



**Software Engineering Department Braude College of Engineering
Final Project – Phase A (61998)**

Cross-Domain Transfer Learning: From Tropical Cyclones to Insect Bite Classification

Bug bite classification using a CNN based TC classification model

CODE: 26-1-R-26

By:

**Ishay Yulzary
Natan Trostianitser**

Advisors:

**Mrs. Elena Kramer
Dr. Dan Lemberg**

Link to GitHub:

<https://github.com/idshay16/Bug-bite-classification-using-a-TC-classification-model>

Contents

1	Introduction	3
2	Background	4
2.1	Medical Background - Bug bites	4
2.2	Tropical Cyclone Patterns	4
2.3	Convolutional Neural Networks	5
2.3.1	Convolutional Layer	5
2.3.2	Pooling layer	6
2.3.3	Fully connected layer	6
2.4	YOLOv3	6
2.5	Transfer Learning	6
2.6	Fine Tuning	7
2.7	Related Work	7
3	Research Process	8
3.1	Medical research	8
3.2	Meteorological research	8
3.3	Model	8
4	Proposed Methodology	9
4.1	Architecture	9
4.1.1	Binary classifier	9
4.1.2	YOLOv3 bug bite detector	9
4.1.3	Multiclass classifier	10
4.2	Potential Modifications	10
5	Engineering Process	12
5.1	Requirements	12
5.2	Data Collection	12
5.3	Expected Challenges	13
5.4	Tools & Technology	13
6	Evaluation	14
6.1	Success Metrics	14
6.2	Process	15
6.3	Software testing	15
6.4	XAI	16
7	Usage Of AI Tools	16
	References	18
8	Appendices	20
8.1	Original Model Architecture	20

Abstract

Insect bites and stings represent a universal health concern, yet accurate diagnosis remains a significant challenge due to the visual ambiguity between distinct arthropod classes and the amorphous nature of skin lesions [10]. This study proposes a novel Cross-Domain Transfer Learning framework that leverages the structural similarity between satellite imagery of Tropical Cyclones (TC) and high-risk insect bites. We hypothesize that the geometric features of cyclones, such as the distinct “eye” and radiating cloud bands, provide a superior feature prior for detecting necrotic centers and concentric erythema compared to standard semantic transfer learning sources [9].

Our methodology adapts a Hybrid Cyclone Intensity Estimation Framework, originally designed for the North Indian Ocean, to distinguish between five vector categories: Mosquito, Flea/Bed Bug, Hymenoptera, Tick, and Spider. The proposed architecture integrates a binary classifier for initial filtering, a YOLOv3-based detector for localization [11], and a multiclass Convolutional Neural Network (CNN) backbone for granular classification. To modernize the underlying architecture, we investigate replacing legacy backbones with ConvNeXt and Swin Transformers [8, 7]. Furthermore, we incorporate Explainable AI (XAI) techniques to ensure the model relies on relevant biological markers rather than background artifacts, thereby providing a transparent decision-support tool for medical professionals.

Keywords: Domain Adaptation, Transfer Learning, Deep Learning, Computer Vision, Convolutional Neural Networks (CNN), Tropical Cyclone, Meteorology, Remote Sensing, Satellite Imagery, Insect Bite, Dermatology, Skin Lesion Classification, Medical Imaging, Geometric Prior, Shape Invariance, Morphological Features, Feature Alignment, Cross-Domain Analysis, Multiclass Classification.

1 Introduction

Insect bites and stings are a universal health concern, ranging from minor annoyances to life-threatening injuries. While most cases resolve independently, severe bites, including those from necrotic spiders or vectors of disease, can lead to rapid tissue death, systemic infection, and long-term scarring. Despite these risks, diagnosing the specific cause of a bite remains a surprisingly difficult and manual process. Medical professionals often struggle to distinguish between a benign mosquito bite, an early-stage tick bite, and a dangerous spider bite, as distinct insect classes can appear visually similar¹ in their early stages. This ambiguity forces physicians to rely on subjective visual intuition or adopt a “watch and wait” approach. This diagnostic bottleneck delays targeted treatment (e.g., antivenom or antibiotics) and strains medical resources, highlighting a clear, urgent need for an automated, high-precision tool capable of classifying the specific insect vector from a standard image.

To address this challenge, this project looks beyond traditional medical imaging. Most standard computer vision models are trained on distinct, rigid objects and struggle with the amorphous, fading boundaries of skin lesions. However, we identified a robust parallel in the field of **Meteorology**, specifically in the analysis of **Tropical Cyclones**. When viewed from a satellite, a tropical cyclone shares a striking morphological similarity to the lesions caused by specific high-risk insects: the distinct necrotic center of a spider bite mirrors the intense “eye” of a cyclone, while the expanding radial patterns of a tick bite (Erythema Migrans) mirror the radiating cloud bands of a storm.

Based on this insight, we propose a novel cross-domain approach. While transferring knowledge between such distinct fields is unconventional, it is not without precedent; for example, researchers have successfully adapted neural networks designed for satellite road imagery to entirely different pattern recognition tasks[1]. We have adapted the *Hybrid Cyclone Intensity Estimation Framework* developed by Mawatwal et al. (2024)[9], a deep learning architecture originally designed to analyze storm structures in the North Indian Ocean. By retraining this model on medical imagery, we leverage its specialized capacity for detecting radial and swirling anomalies to perform **fine-grained insect classification**. We defined five representative classes: **Mosquito Bites**², **Flea/Bed Bug Bites**³, **Hymenoptera Stings**⁴, **Tick Bites**⁵, and **Spider Bites**⁶. This approach re-purposes the meteorological model’s feature extractors, originally built to analyze swirling cloud patterns, to identify the unique visual “signature” of each insect, providing an automated diagnosis of the biting vector.

To ensure the reliability of the transfer learning adaptation, the model will be fine-tuned specifically for the nuance of skin-related textures and skin-tone variations. The model’s performance will be validated using multi-metric evaluation suite, including precision, recall and F1-scores, to ensure diagnostics accuracy remains high across all classes. Furthermore, because the architecture was originally developed for meteorological data, we will integrate XAI⁷ techniques to provide transparency into the model’s decision-making process. By visualizing the model specific areas in the image that influence the decision, This allows us to observe whether the model is truly relying on meaningful visual patterns in the

¹Red, swollen, and inflamed.

²Baseline benign.

³Clustered patterns.

⁴Acute central punctuation.

⁵expanding radial patterns.

⁶Necrotic centers.

⁷Explainable AI

images. The goal of this research is to examine whether patterns learned from tropical cyclone data can be transferred to the automated classification of insect-related skin lesions, with the potential to support accurate and reliable decision-making for medical professionals.

2 Background

2.1 Medical Background - Bug bites

The term bug bites is commonly used to denote both bites and stings inflicted by members of the "phylum Arthropoda"⁸.

The four medically significant classes of arthropods—Chilopoda, Diplopoda, Insecta, and Arachnida—impact human health through four primary pathophysiologic mechanisms.

First, mechanical injury from the bite or sting causes immediate pain and swelling, creating a portal of entry for bacteria that can lead to secondary infections. Second, allergic responses to salivary antigens are common, ranging from localized itching and rashes to life-threatening anaphylaxis. Third, many species are known for injecting toxic venom, the fourth and most significant threat is their role as vectors that transmit clinically important diseases. Whether acting as parasites, venomous threats, or disease carriers, these arthropods pose a complex challenge to human health through the combination of direct tissue damage and systemic biological impact.[10]

2.2 Tropical Cyclone Patterns

The term tropical cyclone refers usually to a class of cyclonic-ally-rotating, low pressure weather systems that has areas of deep cumulus convection⁹ that form generally over warm tropical oceans. Each system is classified according to their intensity, in other words each of them is defined by the maximum continuous wind speed near the surface.

When the intensity is around 17 m/s to 32 m/s, the term tropical storm is used. typically some forecast offices also classify into categories to indicates the potential winds damage that storms inflict. commonly the strongest winds in a tropical cyclone can be found in a ring some distance from the center and there is a calm region near the center, or what can be called the eye of the storm, where the intensity of the wind is lighter. [12]

⁸Major group of animals that includes insects, spiders, and crustaceans, characterized by having jointed legs, segmented bodies, and a hard outer skeleton.

⁹The process where warm, moist air rises in unstable atmospheric conditions, cooling and condensing as it ascends to form fluffy, heaped clouds.

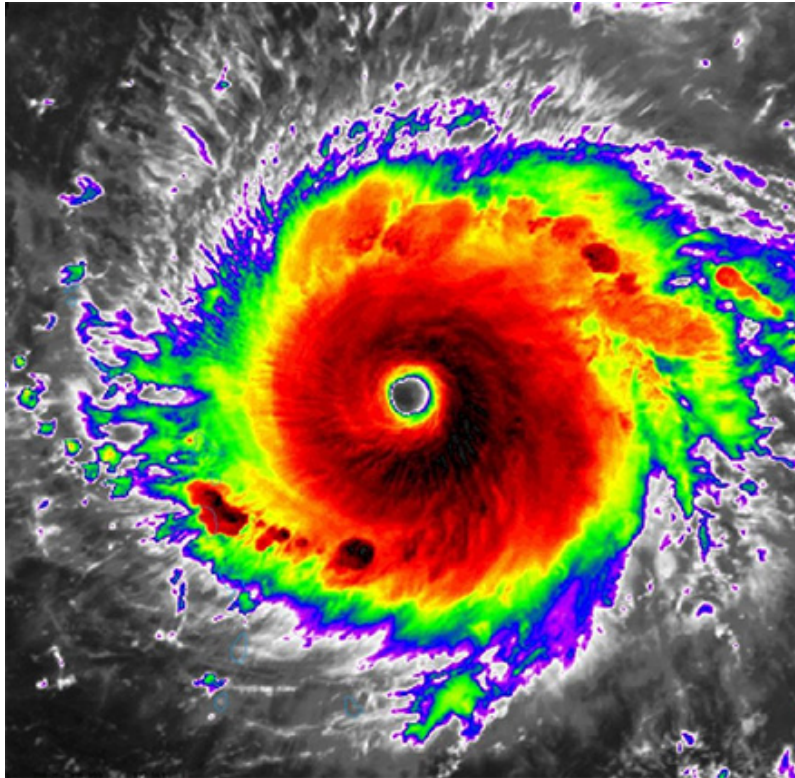


Figure 1: A tropical storm with an "eye" in the center

The eye is surrounded by a ring of clouds that slope outwards with height. At larger radius from the center, storms typically show spiral bands of deep convective clouds.

2.3 Convolutional Neural Networks

Convolutional neural networks are artificial intelligence systems based on multi-layer neural networks that are meant for identification, recognizing, and classification as well as detection and segmentation of objects in images.

The architecture of a typical convolutional neural network typically based on variation between convolution and pooling layers. The architecture of the network consists of different layers, which include: The input layer, the convolution layer, the pooling layer, the FC layer and the output layer.[2]

2.3.1 Convolutional Layer

This layer comprises a bank of learnable kernels. The operation involves sliding a fixed-size window across the input feature map, extracting local feature patches at specific intervals. Each patch undergoes a tensor product with the weight matrix of the kernel; the resulting vectors are then systematically reorganized to construct a new output tensor.

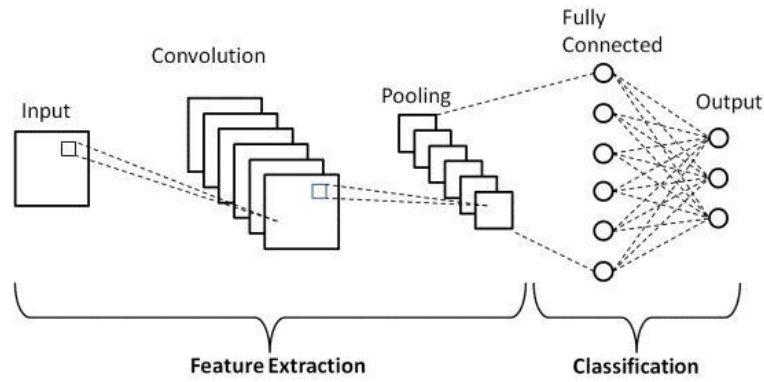


Figure 2: Example of a CNN network

2.3.2 Pooling layer

Commonly referred to as the down-sampling layer, this component typically employs max-pooling, average (mean) pooling, or stochastic pooling. Max-pooling extracts the maximum value within a local neighborhood, while mean-pooling computes the average. Stochastic pooling, by contrast, selects values based on a random probability distribution. By reducing the spatial dimensions of the feature map, these operations significantly decrease the number of network parameters and mitigate the risk of overfitting.

2.3.3 Fully connected layer

Typically situated at the conclusion of a CNN architecture, the Fully Connected (FC) layer transforms the multi-dimensional feature maps into a one-dimensional vector for classification. Mirroring the structure of a Multilayer Perceptron (MLP), it computes outputs through a weighted linear combination of all neurons from the preceding layer, mapping learned features to specific class scores.

2.4 YOLOv3

YOLOv3 formulates object detection as a single regression problem, directly predicting bounding box coordinates and class probabilities from the entire image using a single feed-forward convolutional neural network. By eliminating region proposal generation and feature resampling, all detection stages are integrated into one unified network, resulting in a fully end-to-end detection framework.

The YOLOv3 approach partitions the input image into an $S \times S$ grid. Each grid cell is responsible for detecting objects whose centers lie within it. For every grid cell, the network predicts B bounding boxes, along with their corresponding localization parameters and confidence scores.[11]

2.5 Transfer Learning

Transfer Learning (TL) is a machine learning paradigm designed to overcome the limitations of training models on small or isolated datasets by leveraging knowledge acquired from a related “source” domain. As described by Hosna et al. [3], this approach allows a model to apply previously learned patterns—such as edge detection, texture recognition, and geometric shapes—to a new “target” domain, significantly

reducing the computational cost and data requirements. In the context of this project, we exploit the visual isomorphism between Tropical Cyclones and insect bites; both phenomena manifest as concentric, circular anomalies with distinct gradients radiating from a central point.

To bridge these domains, we utilize **fine-tuning**, a key mechanism discussed in recent transfer learning literature [3]. This technique relies on the observation that the initial layers of deep neural networks learn generic, transferable features (like curves and blobs) regardless of the specific image content. By initializing our model with weights pre-trained on cyclone imagery, we effectively bypass the need for the network to learn basic visual processing from scratch. Instead, we adapt the higher-level layers to reinterpret these shared “circular” features as biological indicators of severity rather than meteorological ones, ensuring robust performance even with a limited dataset of insect bites.

2.6 Fine Tuning

To adapt our pre-trained cyclone model to the bug bite domain, we employ **fine-tuning**, a transfer learning strategy that involves updating the parameters of a pre-trained network on a new dataset. As experimentally demonstrated by Yosinski et al. [14], deep neural networks exhibit a hierarchical structure where early layers capture general, low-level features (such as edges and color gradients) applicable to many visual tasks, while deeper layers capture high-level, domain-specific semantic features.

In our approach, we freeze the initial feature extraction layers of the network, treating them as a fixed “visual cortex” that detects the shared geometric properties of circular anomalies. We then unfreeze and retrain only the final classification layers using our target dataset of insect bites. This strategy allows the model to retain its robust, pre-learned ability to analyze concentric structures while rapidly learning to map those structures to new severity classes, minimizing the risk of overfitting on the smaller target dataset.

2.7 Related Work

Recent literature has established the efficacy of deep learning for automated bite classification, though the transfer learning strategies employed remain largely conventional. Ilijoski et al. [4] proposed an end-to-end system utilizing standard CNN architectures, while Khasanov et al. [5] advanced this field with “DeepBiteNet,” a lightweight ensemble framework designed for efficiency. Both studies rely on **semantic transfer learning**, initializing their models with weights pre-trained on large-scale general datasets (like ImageNet). In this paradigm, the model must expend significant training resources to suppress irrelevant features learned from everyday objects (e.g., vehicles or animals) in order to isolate dermatological patterns specific to insect bites.

Our approach proposes a fundamental shift from semantic to **geometric transfer learning**. Rather than using a generic source domain, we initialize our model using weights trained on Tropical Cyclone imagery. We hypothesize that the structural isomorphism between cyclones and insect bites, both manifesting as concentric, gradient rich circular anomalies, provides a more relevant feature prior than general object datasets. Unlike the related works which rely on the model to learn the concept of a biological reaction from scratch (or adapt from unrelated objects), our method leverages a source domain where the primary visual feature is already the exact geometric shape of interest, potentially offering superior convergence with fewer data samples.

3 Research Process

Our research began with an extensive review of materials, many of which covered topics unfamiliar to us. Through intensive study of these respective fields, we acquired the technical vocabulary necessary to develop a central hypothesis. This led us to conduct a deeper literature search to strengthen our understanding of the disciplines involved in the project. Although mastering this material in a short period was challenging, we successfully identified a significant relationship between the fields. Based on this, we began evaluating potential models earlier than planned in the project timeline (before Phase B), as we recognized that selecting a model early would provide better insight into the core problem.

3.1 Medical research

The research phase prioritized a thorough investigation into the biological and medical aspects of insect identification. Our initial focus was on categorizing insect groups based on distinguishing physical characteristics. We then examined the specific skin reactions caused by different bites; this understanding is vital for guiding the model’s focus toward the most relevant areas of an image.

Because we lack a formal medical background, we encountered challenges with specialized terminology and varying levels of resource quality. Consequently, we adopted a critical approach to our literature review, carefully evaluating each source for reliability. Ultimately, bridging the gap between biological differences and skin manifestations is a prerequisite for our model’s success, provided the underlying data is verified for accuracy

3.2 Meteorological research

To understand the foundation of our base model, we first investigated the recurring patterns in cyclone formation. This required us to define the specific parameters used to identify a cyclone and to understand the various classification systems used in prior research. A primary challenge in this phase was the interpretation of meteorological maps, which are difficult to analyze without a background in atmospheric science. By studying these categorization methods, we have gained the necessary insight to review these maps effectively. However, we remain mindful of the potential for error in recognizing patterns, given the high level of detail required for accurate meteorological analysis.

3.3 Model

To build the proposed system, we first had to conduct an in-depth study of the model’s multi-layered architecture, which consists of binary classification, cyclone detection, and regression modules. We assessed each component to decide which parts required modification to better fit our specific data and geographic focus. We also focused on the core mechanics of transfer learning to ensure our training strategy would lead to accurate predictions. Understanding the underlying architecture is a complex task that involves disassembling and analyzing each functional part. Because the original model was designed for a different application, we must closely oversee the training phase to guarantee it learns the features relevant to our specific field.

4 Proposed Methodology

Our solution utilizes a transfer learning approach, adapting a pre-trained cyclone classification model to identify and categorize insect bites. While the base model was originally designed to analyze tropical cyclones in the Indian Ocean, we are retraining it to distinguish between five specific categories: mosquitoes, fleas/bed bugs, hymenoptera, ticks, and spiders. The primary goal of this cross-domain application is to determine if structural patterns found in meteorological data can be successfully transferred to medical image identification, potentially improving classification accuracy in fields with limited specialized datasets.

4.1 Architecture

Our model is built from 3 parts:

1. A binary classifier.
2. A YOLOv3 bug bite detector.
3. A multiclass classifier.

In addition, as we will list in section 4.2, there are possible modifications to the model that although won't affect the architecture, will have a noticeable effect on its performance.

4.1.1 Binary classifier

This module manages the initial input and image processing. We first apply normalization techniques (rescaling pixel values to $[0,1]$) to ensure the data is clean and suitable for analysis. The processed image is then fed into a four-layer CNN architecture consisting of convolutional blocks with progressively increasing filter counts (32, 64, 128, 128 filters), each followed by 3×3 convolutions, 2×2 max-pooling layers, and dropout regularization (0.2-0.3) to prevent overfitting. The architecture concludes with fully connected layers (Dense 512 with ReLU activation, followed by a single sigmoid output unit) that extract the visual features necessary for classification. The final output of this architecture is a binary decision (probability score) that determines whether the skin lesion contains a bug bite. This serves as a filtering step before the image proceeds to more complex localization and categorization layers.

4.1.2 YOLOv3 bug bite detector

The detection component of our system is based on the YOLOv3 architecture, a powerful one-stage detector known for its balance between speed and accuracy. It uses Darknet-53 as its backbone, employing 3×3 and 1×1 filters along with skip connections to ensure deep feature learning. The core of our approach involves transfer learning: we took a model originally trained to identify cyclones in satellite imagery and retrained its final layers to recognize bug bites in dermatological images. This allows the model to re-purpose high-level spatial pattern recognition capabilities for a new biological context. The model resizes input images using padding to maintain the aspect ratio and predicts bounding boxes around detected bite locations. We refine the final output by filtering out low-probability predictions and

applying Non-Maximum Suppression¹⁰ to eliminate overlapping boxes, ensuring that only the most accurate detections are presented. Also, as we mentioned above in section 4.1, this specific model may be subject to change in the final evaluation version in order to improve model performance

4.1.3 Multiclass classifier

The final stage of the proposed system architecture is the multiclass classification module, which employs an ensemble¹¹ of pre-trained deep learning architectures for robust categorization of detected lesions. This component utilizes transfer learning from multiple CNN models including VGG16, ResNet50, and InceptionV3, originally trained on ImageNet and fine-tuned for dermatological applications. The ensemble approach combines predictions from these diverse architectures to extract comprehensive visual primitives—such as the circularity of a lesion, bite pattern characteristics, and the texture of the surrounding inflammation—and map them to one of five distinct categories: Mosquito, Flea/Bed bug, Hymenoptera, Tick, and Spider.

Each model in the ensemble contributes unique feature representations: VGG16 provides detailed local texture analysis, ResNet50 captures complex spatial relationships through residual connections, and InceptionV3 offers multi-scale feature extraction through its inception modules. The ensemble predictions are combined using weighted voting¹² to achieve superior classification accuracy compared to individual models.

To maintain high diagnostic accuracy, the architecture incorporates stochastic dropout (0.2-0.4) and batch normalization to prevent the network from memorizing specific training samples, thereby ensuring generalization to new, real-world images. The ensemble is optimized using the Adam optimizer with adaptive learning rate scheduling and categorical cross-entropy loss function, which are essential for navigating the complex task of distinguishing between bites that may present with overlapping clinical symptoms.

A key methodology adapted from the original meteorological framework is the use of ensemble learning and image preprocessing during the initial processing phase; by combining multiple model predictions and isolating the primary lesion area while reducing background noise, the classifier can focus exclusively on the morphological characteristics of the bite. This approach leverages the premise that ensemble predictions of distinctive visual features—much like the combined analysis of multiple meteorological indicators—provide the most reliable data points for accurate computer-vision-based classification of dermatological conditions.

Also, as we mentioned above in section 4.1, this specific model may be subject to change in the final evaluation version in order to improve model performance.

4.2 Potential Modifications

The models employed by the original paper we aim to use in our conversion are dated and some are considered obsolete when inspected with a modern lens, in this subsection we will discuss our options for a case where we fail to reach our performance targets when compared to other insect bite classification models. As mentioned above, the existing model uses YOLOv3, ResNet50, VGG-16 and Inceptionv3.

¹⁰NMS eliminates redundant overlapping bounding boxes by keeping only the most confident detection in each region.

¹¹Ensemble methods combine predictions from multiple models to achieve better performance than any individual model.

¹²Weighted voting assigns different importance levels to each model's prediction based on their individual performance metrics.

Our potential modifications to the system can be done because the original training and validation data for the system was included, allowing us to retrain it after selecting different models for the system. Potential replacements we considered are: YOLOv8/YOLOv11 for the detection and annotation since both have better performance and are roughly the same weight (memory-wise) as YOLOv3.

We also plan on upgrading the classification ensemble as the existing models are extremely outdated. We considered multiple options: The first option was selecting models that have the same level of portability as the existing ensemble that are based on more current architectures and DL¹³ paradigms to achieve better performance. The Second option was selecting models that are the new standard of "Heavy Hitters" at the cost of portability the current model has. Another requirement we had for our upgrade options was for the models to be "Plug and Play", i.e. that there is no major refactor or architectural change to the existing solution we are modifying that is needed in order to perform the upgrade. Thus, the models options under consideration are as follows:

Option 1: Lightweight and portable

- **EfficientNetV2-B3** [13] - Replaces InceptionV3; similar parameter count but 7 years newer architecture with faster training and better accuracy.
- **ConvNeXt-Small** [8] - Replaces ResNet50; modernized ResNet design with $2\times$ parameters but significantly better feature extraction.
- **Swin-Small** [7] - Replaces VGG16; transformer-based attention vs old conv stack, comparable parameters but superior performance on spatial tasks.

Option 2: "Heavy Hitters"

- **ConvNeXt-XLarge** [8] - Largest ConvNeXt variant with extreme depth and width.
- **ViT-Giant/22** [15] - Pure transformer architecture processing images as patch sequences.
- **Swin-Giant** [6] - Next-gen hierarchical vision transformer with improved training stability.

In order to give ourselves the best chance of proving our hypothesis, we elected to go with our second option as that will allow us to get the best case scenario that is still somewhat portable (as most cloud providers can serve the required hardware at a very reasonable cost).

¹³Deep Learning

5 Engineering Process

5.1 Requirements

After careful analysis, we have derived a list of requirements for our project. The identified requirements can be seen in Table 1.

#	Description	Type
1	The System shall receive images as inputs	FR
2	the image format will be of a lossless encoding format	NFR
3	The system will output text	FR
4	The output shall contain the confidence value of the model	NFR
5	the system will output the categorization value	FR

Table 1: Requirements

Beyond the primary project goals, additional cases where complementary requirements were considered though in those occasions they were not formulated using strict software engineering conventions and were served as informal guidance during the decision-making process. Hence, we have decided to forgo adding them to this section and they will be explicitly mentioned in the sections in which they are relevant for¹⁴.

5.2 Data Collection

To enable effective transfer learning, a diverse and well-annotated dataset of insect bite images is required. For this purpose, we plan to utilize data from an existing insect bite classification project, DeepBiteNet[5], available on Kaggle under the dataset titled “bug_bite_images” by moonfall_idk. This dataset is organized into three primary subsets: training data, testing data, and a dedicated testing subset containing images of wounds across different skin tones.

The dataset includes multiple categories of insect bites, specifically ants, bed bugs, chiggers, fleas, mosquitoes, spiders, ticks, as well as images containing no visible wounds. For our study, these categories will be adapted and separated into five target classes: fleas, mosquitoes, ticks, spiders, and hymenoptera. This mapping allows alignment between the source dataset and the classification objectives of our model. It is important to note that not all insect bite categories are represented within the different-skin-tone testing subset. Nevertheless, the available samples are sufficient for a preliminary assessment of potential skin-tone-related bias, and no significant bias is observed at this stage.

In addition to the insect bite dataset, we also employ the cyclone dataset used by the original authors of the pre-trained model serving as our source domain. The authors[9] have provided full access to this dataset, including its predefined training and testing splits, which ensures consistency with the original training procedure. All datasets are stored on Google Drive to facilitate efficient access during both training and evaluation phases of the model.

¹⁴can be seen in section 5.5 where we discuss potential modifications

```

bug_bite_dataset/
├── train/
│   ├── fleas/
│   ├── mosquitoes/
│   ├── ticks/
│   ├── spiders/
│   └── hymenoptera/.
├── test/
│   ├── fleas/
│   ├── mosquitoes/
│   ├── ticks/
│   ├── spiders/
│   └── hymenoptera/
├── skin_tone_test/
│   ├── light_skin/
│   └── dark_skin/
└── metadata.csv

```

Figure 3: Directory structure of the insect bite dataset used as the target domain, organized into training, testing, and skin-tone evaluation subsets across the five selected insect classes.

5.3 Expected Challenges

The project faces several anticipated challenges. First, hardware limitations may hinder our progress, as the time required to adjust parameters and re-train the model in each iteration may exceed our initial estimates. Furthermore, we recognize overfitting as a significant risk that could prevent the model from generalizing well to new data. Beyond these technical hurdles, we may face difficulties in model interpretability—specifically, ensuring we can clearly explain the logic behind the model’s decisions within a multi-disciplinary context. Finally, a major constraint is our limited availability and the steep learning curve involved. This makes it challenging to manage the project’s scale, as we must ensure our goals remain realistic and achievable within our timeframe. Balancing these limitations while ensuring the quality of our results will be our primary focus.

5.4 Tools & Technology

The core of this project involves investigating the transferability of a machine learning model—originally trained for cyclone classification—to the domain of infestation and bug bite classification. To support this, our primary development environment will be Google Colab, utilizing T4, H100, or A100 GPUs for training and evaluation, depending on cloud resource availability. For documentation and research, we will use Overleaf for LaTeX typesetting and Google Scholar to source peer-reviewed literature and theoretical background. The technologies utilized in this project are categorized by their application: Model Architecture and Development The project builds upon an original architecture composed of several Convolutional Neural Networks (CNNs). While the baseline utilizes these established structures, we are also considering the integration of Transformers should we decide to modernize the model architecture. The specific models to be evaluated and used include:

- **YOLOv3** (for object detection)
- **VGG16, ResNet50, and InceptionV3** (standard CNN benchmarks)
- **ConvNeXt-XLarge** (modernized CNN)
- **ViT-Giant/22 and Swin-Giant** (Vision Transformers)

Programming Language The system will be developed using Python 3.13. This version was selected to maintain compatibility with the original model’s implementation and to take advantage of the extensive library support available within the Python ecosystem for deep learning.

6 Evaluation

6.1 Success Metrics

In order to evaluate our model, we will use the same metrics that were used in the paper the original model was made for. Since the original paper had made a system that has multiple configurations, of which only one of them is needed for this project, our success metrics will be based on the ”Multiclass Classification” configuration of the model. We will attempt to achieve results that are on par or higher than the ones achieved by the original model. Since the original evaluation metrics are split for each class, we will use an average of the classes metrics as our goal. Said values can be seen on Table 2.

Metric	Score (Avg.)
Precision	0.642
Recall	0.62
F1-Score	0.628

Table 2: Success Metrics

In addition, we also aim to at the very least match the performance of other solutions that currently exist, hence their performance metrics will also be used as a success metric. Those figures will be listed in Table 3. The figures in the table above were the values achieved by the team behind DeepBiteNet[5].

Metric	Score (Avg.)
Precision	0.88
Recall	0.87
F1-Score	0.875
Accuracy	0.846

Table 3: Success Metrics

Furthermore, in the case we decide on modifying the original model by updating the core models, we will also compare the perceived gain in performance compared to the original.

Lastly, we will also try to compute the amount of computing required and see which option is the most ”cost-effective” from a performance to compute perspective.

6.2 Process

The evaluation process will be as follows: First we will split the data into 2 sets: a train set and a test set. Second we will train the model using transfer learning methods in order to teach the model how to classify infestations¹⁵. Third we will run the test set through the model and measure the results, using that we will be able to calculate the performance metrics using the following equations for each metric:

$$precision = \frac{TP}{TP + FP}$$

$$recall = \frac{TP}{TP + FN}$$

$$F1 = \frac{2 \cdot TP}{2 \cdot TP + FP + FN}$$

Since many of these injuries look alike, there's a high risk of overfitting. We'll use the loss function to keep an eye on the gap between our training and validation performance, making sure the model is actually learning to generalize with each epoch.

$$Loss = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

6.3 Software testing

The development process of the proposed system involves modifying existing code components and, potentially, replacing or upgrading different parts of the model architecture. Therefore, a structured testing strategy is required to validate that all components of the system continue to operate as intended throughout development.

First, since the model is fine-tuned to perform transfer learning from cyclone imagery to insect bite classification, regression testing is required. This type of testing ensures that changes made during fine-tuning or further development do not degrade previously achieved performance. By applying regression tests, we can verify that the model maintains its expected behavior or improves upon it after each modification. Second, as alternative models or updated components may be introduced to improve system performance, integration testing is necessary. Integration testing verifies that the different parts of the system—such as the binary classifier, object detection module, and classification components—continue to communicate and function correctly together after changes are applied.

Third, functional testing is used to validate that the system fulfills its intended purpose. This includes confirming that the model can accept skin images as input, detect the presence of a bug bite, localize it within the image, and classify it accordingly. Functional testing ensures that the complete system operates correctly from end to end and that each component contributes as expected to the overall workflow. Finally, acceptance testing is conducted to verify that the system meets the predefined project requirements and overall goals. This testing stage evaluates whether the final system behaves as expected from a

¹⁵Bug Bites. See section 2.1 for more information

user and project perspective, based on agreed-upon success criteria such as correct output format, reasonable prediction confidence, and consistent end-to-end operation. Acceptance testing serves as the final validation step before considering the system complete for its intended academic and experimental use.

6.4 XAI

To ensure the integrity and transparency of the diagnostic system, we integrate an Explainable AI (XAI) framework into the model evaluation stage. Given that the architecture relies on a pre-trained model adapted from the meteorological domain, XAI is essential for verifying that the network prioritizes relevant biological markers—such as the necrotic centers of spider bites or the radial expansion of tick bites—rather than irrelevant background noise or image artifacts. This transparency transforms the model from a “black box” into a verifiable tool for clinical decision support. Our methodology expands upon the foundational work of the base model developers by specifically utilizing LIME (*Local Interpretable Model-agnostic Explanations*) and SHAP (*SHapley Additive exPlanations*) to diagnose model behavior:

- **LIME** will be used to help us understand individual predictions. By masking or changing small parts of a bite image, LIME shows us exactly which pixels the model is looking at. This is important to prove that the model is actually identifying the bite’s features—like the red center or the ring—rather than just picking up on the background or the lighting of the photo.
- **SHAP** values, based on cooperative game theory, will provide a more mathematically consistent global view. By calculating the contribution of each feature to the final prediction, we can confirm the model’s reliability across the entire dataset.

By applying these XAI principles during model evaluation, we can actively identify and mitigate algorithmic biases or dataset imbalances. Furthermore, the insights gained from LIME and SHAP allow for targeted model optimization, refining the architecture based on a deep understanding of why specific classification errors occur. Ultimately, the deployment of XAI serves as a critical quality-control measure, ensuring the model remains robust, ethical, and clinically reliable.

7 Usage Of AI Tools

During our research, we have used multiple AI tools such as: Gemini, ChatGPT, Google Scholar Labs, etc. to aid us in searching for references, writing assistance, and improving the visual aesthetic of the book. AI tools were also integral to us for brainstorming ideas and analyzing the base model’s architecture to see which parts of said model are relevant to us.

We have used many prompts with the AI tools we used as we have worked with said models in a collaborative approach rather than a “Fire and Forget” approach. Hence, you can see in the following figure select example prompts used.

- “Give me a selection of the prompts used in regards to the capstone project”
- “Give me a general example snippet of a bulleted list in LaTeX”
- “How do I add a reference to the book using the bibliography LaTeX library?”

- "Find public datasets for wound analysis or insect bites."
- "Here are the parameters for an image: its width must be up to half a PowerPoint slide. it should be somewhat abstract. it needs to serve as flavor for the presentation and not actually contain data. it needs to revolve around the topic of the capstone."
- "are there any ML models out there that can identify bug bites well? "
- What metrics are most meaningful for medical image classification?

An exact list of all the models that were used in the creation of this project book is:

- ChatGPT - Editorial assistance and idea brainstorming
- Gemini 3 (Pro) - Editorial assistance, LaTeX writing assistance, idea brainstorming and research
- Github Copilot (agent + ask)¹⁶ - Code analysis, Deployment troubleshooting and solution brainstorming
- NotebookLM - Research and Academic article summarization
- Google Scholar Labs - Research

¹⁶Several of the available models were used, some are also listed seperetly in the list

References

- [1] Lutz Berger, Christian Payer, Darko Stern, and Martin Urschler. Cross-domain and cross-dimension learning for image-to-graph transformers. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, pages 1–10, 2025.
- [2] Shuang Cong and Yang Zhou. A review of convolutional neural network architectures and their optimizations. *Artificial Intelligence Review*, 56(3):1905–1969, 2023.
- [3] Asmaul Hosna, Ethel Merry, Jigmey Gyalmo, Zulfikar Alom, Zeyar Aung, and Mohammad Abdul Azim. Transfer learning: a friendly introduction. *Journal of Big Data*, 9(1):102, 2022.
- [4] Bojan Ilijoski, Katarina Trojachanec Dineva, Biljana Tojtovska Ribarski, Petar Petrov, Teodora Mladenovska, Milena Trajanoska, Ivana Gjorshoska, and Petre Lameski. Deep learning methods for bug bite classification: an end-to-end system. *Applied Sciences*, 13(8):5187, 2023.
- [5] Doston Khasanov, Halimjon Khujamatov, Muksimova Shakhnoza, Mirjamol Abdullaev, Temur Toshtemirov, Shahzoda Anarova, Cheolwon Lee, and Heung-Seok Jeon. Deepbitenet: A lightweight ensemble framework for multiclass bug bite classification using image-based deep learning. *Diagnostics*, 15(15):1841, 2025.
- [6] Ze Liu, Han Hu, Yutong Lin, Zhuliang Yao, Zhenda Xie, Yixuan Wei, Jia Ning, Yue Cao, Zheng Zhang, Li Dong, et al. Swin transformer v2: Scaling up capacity and resolution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [7] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021.
- [8] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.
- [9] Manish Kumar Mawatwal and Saurabh Das. An end-to-end deep learning framework for cyclone intensity estimation in north indian ocean region using satellite imagery. *Journal of the Indian Society of Remote Sensing*, 52(10):2165–2175, 2024.
- [10] Jim Powers and Rachel H McDowell. Insect bites. In *StatPearls [Internet]*. StatPearls Publishing, 2023.
- [11] Joseph Redmon and Ali Farhadi. Yolov3: An incremental improvement. *arXiv preprint arXiv:1804.02767*, 2018.
- [12] Roger K Smith and Michael T Montgomery. *Tropical cyclones: Observations and basic processes*, volume 4. Elsevier, 2023.
- [13] Mingxing Tan and Quoc V Le. Efficientnetv2: Smaller models and faster training. *arXiv preprint arXiv:2104.00298*, 2021.
- [14] Jason Yosinski, Jeff Clune, Yoshua Bengio, and Hod Lipson. How transferable are features in deep neural networks? *arXiv preprint arXiv:1411.1792*, 2014.

- [15] Xiaohua Zhai, Alexander Kolesnikov, Neil Houlsby, and Lucas Beyer. Scaling vision transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022.

8 Appendices

8.1 Original Model Architecture

In the following figure there is an image of the model developed by the team behind "An End-to-End Deep Learning Framework for Cyclone Intensity Estimation in North Indian Ocean Region Using Satellite Imagery"[9]

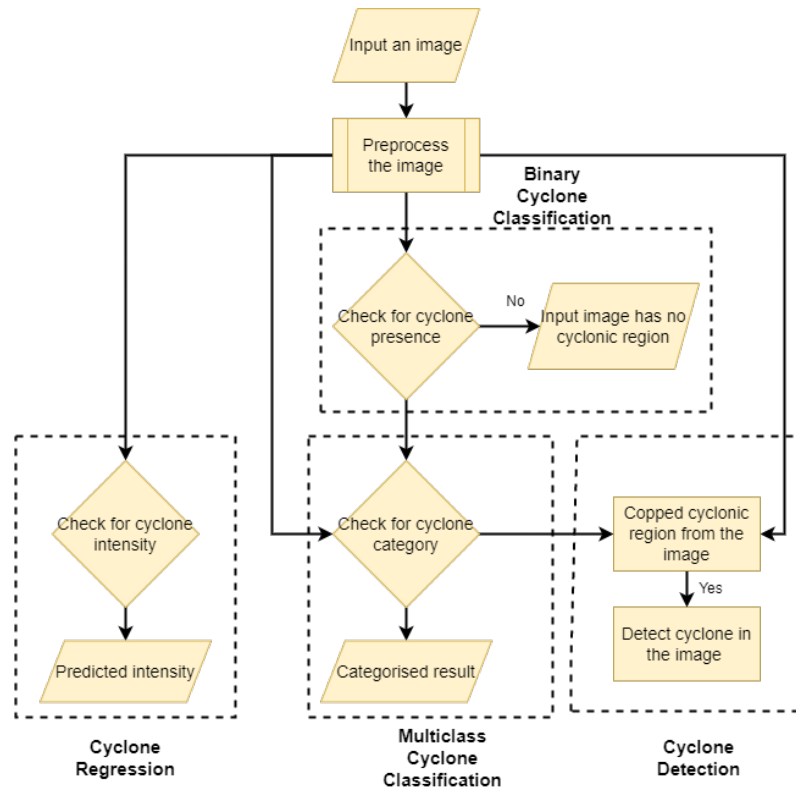


Figure 4: Example of a CNN network